Network Model

- Many OR situations can be modeled and solved as networks (nodes connected by branches i.e. edges). (In CS network models =graph theory)
- The solution of these situations is accomplished through a variety of network optimization algorithms. 4 of these:
    - Minimal Spanning Tree
    - Shortest Route Algorithm
    - Maximal Flow Algorithm
    - Critical Path Method

Minimal Spanning Tree Algorithm:

The minimal spanning tree links the nodes of a network using the smallest total length of connecting branches. A typical application occurs in the pavement of roads linking towns, either directly or passing through other towns.

ALGORITHM 1 Prim's Algorithm.

procedure *Prim*(G: weighted connected undirected graph with n vertices)
T := a minimum-weight edge
for $i := 1$ **to** $n - 2$
  e := an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T
  $T := T$ with e added
return T {T is a minimum spanning tree of G}

Ex:

ALGORITHM 2 Kruskal's Algorithm.

procedure *Kruskal*(G: weighted connected undirected graph with n vertices)
T := empty graph
for $i := 1$ **to** $n - 1$
  e := any edge in G with smallest weight that does not form a simple circuit
    when added to T
  $T := T$ with e added
return T {T is a minimum spanning tree of G}

Ex:

Shortest Route Problem

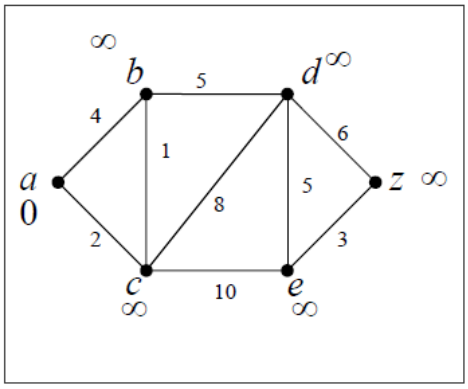
The shortest route problem determines the shortest route between a source and destination in a transportation network.

Dijkstra's algorithm is one of the well-known algorithms for solving shortest route problems.

Dijkstra's Algorithm:

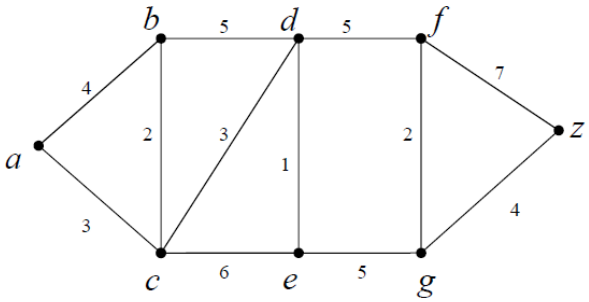
procedure **Dijkstra** (G: positive weighted connected simple graph)
// G has vertices $a = v_0, v_1, \ldots, v_n = z$ and
// weights $w(v_i, v_j)$ where $w(v_i, v_j) = \infty$ if no edge
// initialization
    for (i =1; $i \le n$; i++) $L(v_i) = \infty$
    $L(a) = 0$
    $S = \varnothing$
    while ($z \notin S$) {
        u = a vertex not in S with $L(u)$ minimal
        $S = S \cup \{u\}$
        for $\forall v \notin S$ {
            if ($L(u) + w(u, v) < L(v)$)
                $L(v) = L(u) + w(u, v)$
        }
    }
$L(z)$ = length of the shortest path from a to z
Dijkstra's algorithm has $O(n^2)$ operations (addition and comparison).

Ex:



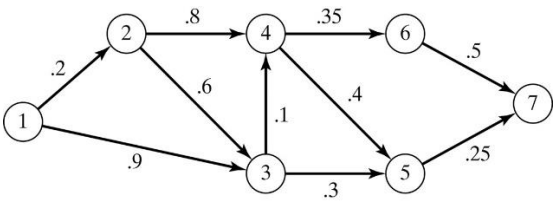
$S=\varnothing$

Ex:



Ex: I.Q. Smart drives daily to work. Smart decides to choose a route that maximizes the probability of not being stopped by police.

The following graph shows the possible routes from home to work, and the associated probabilities of not being stopped on each segment.

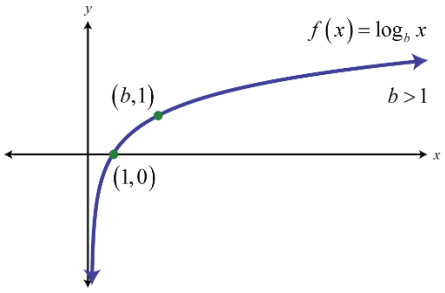


The probability of not being stopped on a route is the product of the probabilities of its segments. For example, the probability of not receiving a fine on the route 1->3->5->7 is 0.9x0.3x0.25 = 0.0675. Smart's objective is to select the route that maximizes the probability of not being fined.

The problem can be formulated as a shortest route model by using logarithmic transformation to convert the probability into the sum of the logarithms of the probabilities.

That is:

p_{1-k} and $log(p_{1-k})$ are both monotone decreasing, thus maximizing p_{1-k} is equivalent to maximizing $log(p_{1-k})$, which in turn is equivalent to minimizing $-log(p_{1-k})$.



LP Formulation of the Shortest Route Problem:

If we add

$$\sum_{i=1}^{|V|} \sum_{\substack{j=1 \\ j \neq i}}^{|V|} w_{ij} x_{ij} \leq W$$

where w_{ij} is the weight of edge from *i* to *j* and W is a limit, then the problem turns out to be the constrained shortest path problem which is NP-Complete.